F-6.5.1

POLICY PARSING & CLAUSE EXTRACTION

Version 1.0 | March 2026 | Phase 1 — MVP Core | Foundational Feature

F-6.5.1	Phase 1	UPSTREAM	Active	T2 Sensitive	Owner: Core Engine	Feeds: 6.5.2, 6.5.5	Security: 8.3, 8.4, 8.6

Feature Role in the Platform

F-6.5.1 is the foundation of the entire coverage intelligence stack. Nothing downstream works without it. Every coverage determination, every claim routing decision, every overlap resolution, and every counterfactual analysis depends on having accurate, structured, clause-level data extracted from uploaded policy documents. This feature does not guess. It does not infer what a policy probably says. It extracts, with confidence scoring, what the document actually contains — and it routes anything it cannot extract with high confidence to human review before it is ever used in guidance.

Accuracy target: 99%+ at the clause level, achieved through multi-pass extraction, canonical taxonomy matching, dual-model validation, and a mandatory human review queue for any clause below the confidence threshold.

1. Feature Identity

Feature ID	F-6.5.1
Feature Name	Policy Parsing & Clause Extraction
Product Bible Location	Section 6.5.1 — Master Feature Catalog > Feature Pages (Top-Level Features)
Phase	Phase 1 (MVP Core — required for launch)
Status	Specified. Not yet implemented.
Criticality	CRITICAL — no downstream feature in Section 6.5 operates without this output
Data Classification	T2 Sensitive (policy documents, financial identifiers, coverage terms)
Primary Owner	Core Parsing Engine (backend service)
Processing Mode	Asynchronous — upload is immediate, extraction runs as background job

1.1 — User Problem Solved

A traveler has purchased a travel insurance policy. The policy document is a 40-page PDF written by lawyers for underwriters, not for travelers. Somewhere in that document are the specific clauses that will determine whether a delayed bag is covered, which carrier the traveler must file with first, what the time window is for filing a claim, and what documentation they must produce. The traveler does not know where those clauses are. Most travelers never find them.

F-6.5.1 solves this by taking that PDF and producing a structured, machine-readable, clause-level extraction — every coverage category, every exclusion, every time limit, every documentation requirement — organized, tagged, and ready for the Coverage Graph, the Claim Routing engine, and the traveler-facing guidance surfaces.

The traveler uploads a document once. Everything else on the platform becomes possible.

1.2 — What This Feature Is / Is Not

What This Feature IS	What This Feature IS NOT
A structured clause extraction pipeline that converts unstructured PDF policy text into machine-readable, confidence-scored coverage objects.	A legal interpretation service. It extracts what the document says, not what it means in a legal dispute.
A source-citation-bound system: every extracted clause references the exact text block and page number it came from.	A guarantee of coverage. Extraction accuracy does not equal coverage entitlement. Insurers make final determinations.
A confidence-scored output: every clause has a confidence level. Low-confidence clauses go to human review before use.	A replacement for reading the policy. The platform helps users navigate; it does not substitute for their informed review.
An accuracy-optimized system with multi-pass extraction, dual-model validation, and canonical taxonomy matching.	An OCR-only service. OCR is one input layer. The extraction pipeline has seven distinct stages beyond OCR.
A versioned archive: when a policy document is updated, the previous version is preserved with its extractions intact.	A real-time extraction service. Processing is asynchronous. The upload confirms immediately; extraction completes in the background.
A hallucination-guarded system: if a clause is not extractable with sufficient confidence, it is marked as UNEXTRACTED — never inferred.	A catch-all for every policy type on the market. Supported policy types and clause taxonomies are explicitly defined (see Section 4).

2. Accuracy Architecture — How 99%+ Is Achieved

The 99%+ clause-level accuracy target is not aspirational — it is an engineering constraint. Every architectural decision in this feature is made in service of this target. This section documents how accuracy is built, not claimed.

2.1 — The Accuracy Problem

Insurance policies are adversarially difficult documents to parse:

- Variable structure: no two insurers use the same section headings, page layout, or clause ordering
- Cross-references: exclusion clauses refer to benefit clauses by section number, which may vary between policy editions
- Ambiguous language: "reasonable expenses" or "covered event" are defined differently in each policy and often appear as assumed terms
- Multi-column layouts, footnotes, sidebars: OCR engines frequently misorder text from complex layouts
- Scanned PDFs: lower-quality scans degrade OCR accuracy, introducing character substitution errors
- Amendment sheets: policies may have separate endorsement pages that modify base coverage — these must be detected and merged
- Credit card benefit guides: formatted as marketing materials, not legal documents — different structure requires different extraction logic

A single misextracted exclusion clause could lead a traveler to file with the wrong carrier, miss a filing window, or believe they have coverage they do not. The consequences of extraction error are asymmetrically bad. The architecture reflects this.

2.2 — Seven-Stage Extraction Pipeline

Every policy document passes through a deterministic seven-stage pipeline. Each stage has explicit pass/fail criteria. A failure at any stage produces a structured failure code — not a degraded output — and routes to the appropriate remediation path.

#	Stage	Input → Output	How Accuracy Is Protected
1	**Document Intake & Validation**	*Raw upload → validated document object*	File type validation (PDF, image). Size limits enforced. Virus/malware scan before any processing. Hash (SHA-256) computed and stored — any document modification is detectable. Corrupt or password-protected PDFs are rejected with an explicit error, never silently processed with degraded output.
2	**Layout Analysis & Pre-Processing**	*PDF → structured page layout objects*	PDF structure tree is parsed first (for tagged PDFs). For untagged PDFs, layout detection identifies columns, headers, footers, sidebars, footnotes, and tables as distinct zones. Each zone is classified before OCR is applied, ensuring text is extracted in reading order, not pixel order.
3	**OCR Execution (Conditional)**	*Page images → raw text strings*	OCR is applied only where the PDF does not contain embedded selectable text. For native PDFs, text is extracted directly (higher fidelity). OCR confidence scores are captured per character block from the OCR engine. Blocks below 85% character-level confidence are flagged for secondary OCR pass with a different engine (two-engine consensus model).
4	**Text Normalization & Clause Boundary**	*Raw text → clause candidate list*	Legal citation markers, section numbers, cross-reference patterns, and definition markers are identified via rule-based pattern library (2,400+ patterns covering

	Detection		95% of observed policy structures). Clause boundaries are detected using these patterns plus learned structural signals. Each clause candidate includes: raw text, page number, zone type, position within document.
5	**Semantic Classification Against Canonical Taxonomy**	*Clause candidates → classified clause objects*	Each clause candidate is matched against the Canonical Clause Taxonomy (see Section 4). Primary classification is done by a fine-tuned classification model trained on labeled insurance policy text. A second independent model validates the primary classification. Agreement = high confidence. Disagreement = medium confidence → human review queue. Both models fail to classify = UNEXTRACTED.
6	**Relationship & Cross-Reference Resolution**	*Classified clauses → linked clause graph*	Exclusions are linked to the benefit clauses they modify. Defined terms are resolved across the document (e.g., "covered event" is traced to its definition). Amendment sheets are detected and their modifications applied to base clauses with explicit change markers. Time limits are extracted and normalized to a canonical format (e.g., "within 21 days" → {value:21, unit:"days", reference:"incident_date"}).
7	**Confidence Scoring & Output Finalization**	*Clause graph → final PolicyExtractionResult*	Every clause in the output receives a composite confidence score (0.0–1.0) computed from: OCR confidence of source text, taxonomy match confidence, cross-reference resolution success, and semantic coherence score. Clauses above CONFIDENCE_THRESHOLD_HIGH (0.90) are marked AUTO_ACCEPTED. Clauses between CONFIDENCE_THRESHOLD_MEDIUM (0.70) and 0.90 are marked PENDING_REVIEW. Clauses below 0.70 are marked UNEXTRACTED. An UNEXTRACTED clause is never used in guidance without human resolution.

2.3 — Dual-Model Validation Architecture

The core of the accuracy architecture is the dual-model validation at Stage 5. This is not fine-tuning the same model twice — it is using two architecturally distinct models so their error patterns do not correlate.

Model A (Primary)	Fine-tuned transformer model trained on labeled insurance policy clause corpus. High recall, lower precision on ambiguous clauses. Produces: clause_type, confidence, supporting_text_span.
Model B (Validator)	Rule-augmented classification model incorporating the 2,400+ pattern library as explicit features. Higher precision on well-structured policies, lower recall on scanned/degraded documents. Produces: clause_type, confidence, pattern_match_evidence.
Agreement rule	If Model A and Model B agree on clause_type: composite confidence = max(A_conf, B_conf) × 1.05 (agreement bonus, capped at 1.0).
Disagreement rule	If Model A and Model B disagree on clause_type: composite confidence = min(A_conf, B_conf) × 0.80 (penalty). Record routed to CLAUSE_REVIEW_QUEUE with both classifications presented to reviewer.
Both fail	If neither model produces a classification above their individual minimum threshold (0.40): clause is marked UNEXTRACTED. Source text is preserved. Reviewer sees the raw text and must manually classify or confirm as non-extractable.
GPU isolation	Both models run in isolated compute context. Neither has access to user data outside the current extraction job. Model version IDs are logged with every extraction per S-INV-08 (Section 8.3.7 of security spec).

2.4 — Confidence Threshold System

Confidence Band	Score Range	Status	Behavior
HIGH	**0.90 – 1.00**	**AUTO_ACCEPTED**	Clause is included in PolicyExtractionResult. Available to Coverage Graph and all downstream features immediately. Source citation attached. No human review required.
MEDIUM	**0.70 – 0.89**	**PENDING_REVIEW**	Clause is placed in clause_review_queue with priority = STANDARD. It is excluded from downstream use until a human reviewer accepts, rejects, or corrects it. The extraction result is marked PARTIAL until all PENDING_REVIEW clauses are resolved.
LOW	**0.40 – 0.69**	**PENDING_REVIEW (Priority: HIGH)**	Clause is placed in clause_review_queue with priority = HIGH. Excluded from downstream use. Founder notified via FOCL Decision Queue if HIGH priority items accumulate (>5 unresolved for a single document after 24 hours).
UNEXTRACTED	**0.00 – 0.39**	**UNEXTRACTED**	Source text is preserved in the extraction record. Clause is logged as UNEXTRACTED. Traveler guidance surfaces display the UNEXTRACTED indicator for that coverage category (never silence — see Section 7.2 guidance doctrine). Reviewer may manually classify.

2.5 — The Hallucination Guardrail (Non-Negotiable)

- **Hallucination Guardrail — F-6.5.1 Doctrine**
  - Every extracted clause must cite the exact text span from the source document that supports the extraction. No citation = no extraction.
  - If the policy document does not contain a clause matching a taxonomy category, that category is marked ABSENT — not UNEXTRACTED. ABSENT means the feature is confirmed not present. UNEXTRACTED means we could not determine presence or absence.
  - The system never infers what a policy probably covers based on the policy type, issuer, or similar policies. Only what the submitted document contains.
  - No clause may be promoted to AUTO_ACCEPTED status without a supporting_text_span that can be verified against the source document.
  - Cross-reference resolution (e.g., linking an exclusion to a benefit) is logged. If the cross-reference cannot be resolved, it is marked UNRESOLVED — never silently dropped.
  - These rules are enforced at the output schema level (NOT NULL constraints on supporting_text_span, source_page, source_zone) — not merely by convention.

2.6 — Version Control & Drift Detection

Policies change. An insurer may issue an updated policy document that modifies coverage terms. The platform must detect these changes and re-extract, not silently serve stale data.

New version detection	When a user uploads a document with the same insurer and policy number as an existing document, the system computes a similarity score. If the content has changed (>5% structural difference or any change to a previously extracted clause), a new version is created.
Diffing	The extraction result for the new version is compared clause-by-clause against the previous version. Changes are classified: EXPANDED (new coverage added), NARROWED (coverage reduced or exclusion added), CLARIFIED (language

	changed, coverage interpretation unchanged), RESTRUCTURED (section numbering changed, content same).
User notification	If a policy update narrows coverage relative to the previous version, the traveler is notified with a plain-English explanation. See Section 7.2 (Calm Mode Language System) for notification language rules.
Historical preservation	Previous version extractions are preserved indefinitely (subject to 8.2 retention schedule). The Coverage Graph can reconstruct what coverage looked like at any point in time — required for claim dispute context.
Immutable version anchor	Every extraction result is assigned a policy_version_id. All downstream objects (coverage nodes, routing recommendations, counterfactual evaluations) reference this version ID. If the version changes, downstream objects are flagged for re-evaluation.

3. Supported Document Types & Input Handling

3.1 — Supported Document Types (Phase 1)

Document Type	Format(s)	Phase 1 Extraction Scope
Travel Insurance Policy	*PDF (native or scanned)*	Full clause extraction: benefits, exclusions, time limits, documentation requirements, definitions, endorsements, geographic scope, per-person and per-trip limits, deductibles, coordination of benefits language. All clause taxonomy categories applicable.
Credit Card Travel Benefit Guide	*PDF, HTML-to-PDF conversion*	Benefit extraction: travel delay, trip cancellation, baggage, rental car, emergency medical. Cardholder eligibility rules, purchase requirements, coverage trigger conditions, claims contact information. Note: benefit guides frequently update — refresh cadence rules apply (see 3.3).
Insurance Certificate / Summary of Benefits	*PDF*	Summary-level extraction. These are not the policy contract — they are extracted at a lower confidence threshold and always displayed to the user with a note that the full policy document is the controlling document.
Policy Endorsement / Amendment Sheet	*PDF (often single-page)*	Detected as amendment type. Modifications are applied to the base policy extraction with explicit change markers. If the base policy is not yet extracted, the amendment is queued pending base policy upload.
Confirmation of Coverage Letter	*PDF*	Structured extraction of: coverage period, covered travelers, policy number, insurer contact. Limited clause extraction — these documents confirm coverage but do not define terms.

3.2 — Unsupported Document Types (Phase 1)

The following document types are NOT processed by F-6.5.1 in Phase 1

- Scanned documents with resolution below 150 DPI — rejected with explanation and resolution guidance for user
- Password-protected PDFs — rejected with instructions to save an unprotected copy
- Documents in languages other than English — stored, not processed; extraction queued for Phase 2 (multilingual extraction)
- Health insurance policies — out of scope for the travel coverage graph; stored in vault only per 8.3
- Home/renters/auto insurance policies — not in Phase 1 taxonomy (added in Phase 2)
- Handwritten documents — rejected; OCR cannot achieve required confidence level
- Documents larger than 100MB — rejected; user prompted to check if the file is corrupted

3.3 — Credit Card Benefit Guide Refresh Cadence

Credit card benefit guides are issued and updated by card issuers on irregular schedules. Unlike user-uploaded policy documents, platform-maintained benefit guide versions must be actively refreshed to stay current.

Platform-maintained guides	The platform maintains an internal library of benefit guides for major travel credit cards. These are extracted, versioned, and served to users who indicate they hold those cards. Extraction is performed when a new version is detected or on the standard quarterly refresh cycle.
Quarterly refresh cycle	Platform-maintained guides are re-fetched and compared against the stored version every 90 days. If content has changed, a new version is extracted and the FOCL Decision Queue is notified.
User-uploaded guides	A user may upload their own benefit guide (e.g., a newer version than the

	platform has). This is processed identically to a policy document upload and becomes the active version for that user's account.
Staleness indicator	If a platform-maintained guide has not been refreshed within 120 days, all coverage outputs derived from it display a staleness indicator: "Benefit information last verified [date]. Verify with your card issuer."
Source citation preservation	All benefit information, regardless of source, cites the specific section of the benefit guide it was extracted from. The citation is displayed to the traveler when coverage information is shown.

3.4 — Upload Handling & File Security

Per Section 8.3 (Encryption & Key Management), all uploaded policy documents are T2 Sensitive data. Upload handling must comply with the object storage security controls defined there.

Upload initiation	Client requests a presigned upload URL from the backend. URL is scoped to the user's storage prefix only. TTL: 15 minutes. Upload proceeds directly to object storage (client never touches the application server).
Pre-processing security	File is scanned for malware/malicious content before extraction pipeline is initiated. SHA-256 hash is computed from the raw uploaded bytes. Hash is stored in policy_documents table.
Async queuing	On upload confirmation (chunked upload complete + storage confirmed), a policy_parse_requested job is inserted into the job_queue table (existing schema). This is the handoff from the upload flow to the extraction pipeline.
Progress tracking	The user sees: Uploaded → Processing → Complete (or Needs Review). They are never shown a "Complete" status until extraction has finished and all HIGH confidence clauses are available. PENDING_REVIEW status is shown as "Almost done — a few sections are being verified."
Corruption detection	If the SHA-256 hash of the extracted text does not match expectations from the layout analysis (e.g., text extracted is implausibly short for document size), a corruption flag is set and the document is routed to manual review rather than producing a low-confidence extraction.

4. Canonical Clause Taxonomy

The Canonical Clause Taxonomy is the controlled vocabulary for all clause types the platform recognizes. Every extracted clause is assigned a clause_type from this taxonomy. The taxonomy is the shared language between F-6.5.1 (extraction), F-6.5.2 (Coverage Graph), F-6.5.5 (Claim Routing), and all downstream features. No ad-hoc clause types are permitted in the production pipeline.

4.1 — Taxonomy Family Structure

FAM	Family Name	Downstream Consumer	Description
FAM-01	**Trip Cancellation & Interruption**	*F-6.5.2, F-6.5.5*	Clauses governing coverage when a trip is cancelled before departure or interrupted after departure. Includes covered reasons, maximum benefit amounts, exclusions, and documentation requirements.
FAM-02	**Travel Delay & Missed Connection**	*F-6.5.2, F-6.5.5, F-6.5.8*	Clauses governing coverage for delays beyond specified time thresholds and missed connections resulting from covered events. Includes minimum delay threshold, per-day limits, aggregate limits, and covered expenses.
FAM-03	**Baggage & Personal Effects**	*F-6.5.2, F-6.5.5*	Coverage for lost, stolen, or damaged baggage. Includes per-item limits, aggregate limits, high-value item sub-limits, exclusions for specific item categories, and claim documentation requirements.
FAM-04	**Baggage Delay**	*F-6.5.2, F-6.5.5*	Separate from loss — covers essential item purchases when checked baggage is delayed beyond a threshold. Includes delay threshold, per-day limit, outbound-only vs. return-trip applicability.
FAM-05	**Emergency Medical & Dental**	*F-6.5.2, F-6.5.5*	Coverage for medical expenses incurred during the trip. Includes aggregate limit, per-condition sub-limits, pre-existing condition exclusions and lookback periods, dental sub-limits, and ambulance coverage.
FAM-06	**Emergency Medical Evacuation & Repatriation**	*F-6.5.2, F-6.5.5, F-6.6.4*	Coverage for medically necessary evacuation and repatriation of remains. Includes aggregate limit, carrier-selection rules, and coordination with medical coverage.
FAM-07	**Non-Medical Evacuation**	*F-6.5.2, F-6.5.5*	Coverage for evacuation due to security threats, natural disasters, or government-ordered evacuations. Distinct from medical evacuation. Includes trigger conditions, aggregate limit, and approved carrier rules.
FAM-08	**Travel Accident & AD&D**	*F-6.5.2*	Coverage for accidental death and dismemberment occurring during travel. Includes benefit schedule, common carrier vs. all-travel applicability.
FAM-09	**Rental Car Coverage**	*F-6.5.2, F-6.5.5*	Coverage for collision damage or theft of a rental vehicle. Includes per-incident limit, excluded vehicle types, purchase requirement (for card benefits), and CDW/LDW waiver rules.
FAM-10	**Pre-Existing Condition Handling**	*F-6.5.2, F-6.5.5*	Waiver conditions, lookback period definition, stability requirement. Critical for accurate medical coverage determination. Clause type is extracted separately and linked to FAM-05 and FAM-06.
FAM-11	**General Exclusions**	*F-6.5.2, F-6.5.5*	Exclusions applying across multiple coverage categories: war/civil unrest, intentional acts, substance use, participation in hazardous activities. Each exclusion is linked to the coverage families it modifies.
FAM-12	**Filing Requirements & Time Limits**	*F-6.5.5, F-6.5.14*	Documentation requirements, filing windows, proof of loss deadlines, notice requirements. Extracted and normalized to machine-readable format:

			{window_value, window_unit, reference_event, documentation_list}.
FAM-13	**Definitions**	*F-6.5.2, all families*	Policy-specific definitions of key terms (covered event, family member, trip, common carrier, etc.). Used for cross-reference resolution. Every defined term in the taxonomy families is traced to its definition.
FAM-14	**Coordination of Benefits & Primacy**	*F-6.5.5*	Rules governing the sequence of coverage when multiple policies apply. Which policy pays first, subrogation rights, claim-filing-with-primary-carrier requirements.
FAM-15	**Authority & Government Action**	*F-6.5.12*	Coverage for disruptions caused by government-issued travel advisories, border closures, visa denials, or security alerts. Distinct from FAM-07 in trigger mechanism.

4.2 — Clause-Level Object Structure

Each extracted clause becomes an ExtractedClause object with the following canonical fields:

```
ExtractedClause {
  clause_id:            uuid          -- stable identifier for this clause
  policy_version_id:    uuid          -- which version of the policy this came from
  clause_type:          ClauseType    -- enum from Canonical Taxonomy (FAM-01 through FAM-15)
  clause_subtype:       text          -- more specific classification within family
  extraction_status:    ExtractionStatus  -- AUTO_ACCEPTED | PENDING_REVIEW | UNEXTRACTED | ABSENT
  confidence_score:     decimal(4,3)  -- 0.000 to 1.000
  model_a_type:         text          -- Model A classification (preserved even if overridden)
  model_a_confidence:   decimal(4,3)
  model_b_type:         text          -- Model B classification
  model_b_confidence:   decimal(4,3)
  model_version_a:      text          -- version ID of Model A used (per S-INV-08)
  model_version_b:      text          -- version ID of Model B used
  supporting_text_span: text          -- NOT NULL: verbatim source text (hallucination guard)
  source_page:          int           -- NOT NULL: page number in source document
  source_zone:          text          -- NOT NULL: zone type (body|header|footer|table|footnote)
  source_position:      jsonb         -- bounding box {x1,y1,x2,y2} if available
  structured_value:     jsonb         -- machine-readable extraction (amounts, limits, windows)
  cross_references:     uuid[]        -- links to related clause_ids (e.g., exclusion → benefit)
  unresolved_references:jsonb         -- cross-references that could not be resolved
  reviewer_id:          uuid          -- null until reviewed; set on PENDING_REVIEW resolution
  reviewer_action:      ReviewAction  -- ACCEPTED | CORRECTED | REJECTED
  reviewer_notes:       text
  reviewed_at:          timestamptz
  created_at:           timestamptz   NOT NULL
  updated_at:           timestamptz   NOT NULL
}
```

5. Database Schema — New Migrations Required

The existing database schema (Migrations 00001 and 00002 + Governance Substrate v1) does not include the tables required for F-6.5.1. The following migrations must be applied in sequence. They are designed to be additive only — no existing tables are modified.

Migration M-F651-01: Core Policy & Extraction Tables

```
-- ====================================================================
-- MIGRATION M-F651-01: Policy Parsing & Clause Extraction Schema
-- Feature: F-6.5.1
-- Depends on: 00001 (initial schema), governance_substrate_v1
-- ====================================================================

-- ENUMS

CREATE TYPE policy_document_status AS ENUM (
  'uploaded',          -- document received, not yet processed
  'queued',            -- job inserted into job_queue
  'processing',        -- extraction pipeline running
  'partial',           -- extracted but has PENDING_REVIEW clauses
  'complete',          -- all clauses AUTO_ACCEPTED
  'needs_review',      -- has LOW confidence or unresolved clauses
  'failed',            -- extraction failed; see failure_code
  'unsupported'        -- document type not in supported list
);

CREATE TYPE policy_document_type AS ENUM (
  'travel_insurance_policy',
  'credit_card_benefit_guide',
  'certificate_of_insurance',
  'policy_endorsement',
  'confirmation_of_coverage',
  'unknown'
);

CREATE TYPE extraction_status AS ENUM (
  'AUTO_ACCEPTED',   -- confidence >= 0.90, ready for downstream use
  'PENDING_REVIEW',  -- confidence 0.40-0.89, human review required
  'UNEXTRACTED',     -- confidence < 0.40, source text preserved
  'ABSENT'           -- confirmed not present in document
);

CREATE TYPE review_action AS ENUM (
  'ACCEPTED',    -- reviewer confirms extraction is correct
  'CORRECTED',   -- reviewer modified the extraction
  'REJECTED'     -- reviewer marks extraction as invalid
);

CREATE TYPE version_change_type AS ENUM (
  'EXPANDED',      -- coverage broadened
  'NARROWED',      -- coverage reduced / exclusion added
  'CLARIFIED',     -- language changed, substance same
  'RESTRUCTURED',  -- structure changed, substance same
  'NEW'            -- clause did not exist in previous version
  'REMOVED'        -- clause present in previous version, absent now
);

-- ---------------------------------------------------------------------
-- policy_documents: One row per uploaded source document
-- Encryption: object stored in SSE-KMS bucket (8.3). Row stores metadata only.
-- ---------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS policy_documents (
  id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id            uuid NOT NULL REFERENCES auth.users(id),
  trip_id            uuid REFERENCES trips(id),   -- nullable; linked when trip exists
  document_type      policy_document_type NOT NULL DEFAULT 'unknown',
  status             policy_document_status NOT NULL DEFAULT 'uploaded',
  failure_code       text,
  original_filename  text NOT NULL,
  storage_key        text NOT NULL,   -- object storage key (never the URL)
```

```
  storage_bucket      text NOT NULL,
  file_size_bytes     bigint NOT NULL,
  mime_type           text NOT NULL,
  hash_sha256         text NOT NULL,   -- integrity check; detects modification
  page_count          int,             -- populated after layout analysis
  issuer_name         text,            -- extracted: insurer or card issuer name
  policy_number       text,            -- encrypted field; see 8.3.4
  policy_number_encrypted text,        -- ciphertext of policy_number
  effective_date      date,
  expiration_date     date,
  is_platform_maintained boolean NOT NULL DEFAULT false,
  refresh_cadence_days int,            -- for platform-maintained guides
  last_refreshed_at   timestamptz,
  metadata            jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at          timestamptz NOT NULL DEFAULT now(),
  updated_at          timestamptz NOT NULL DEFAULT now(),

  CONSTRAINT policy_documents_filename_not_empty CHECK (length(trim(original_filename)) > 0),
  CONSTRAINT policy_documents_hash_format CHECK (length(hash_sha256) = 64),
  CONSTRAINT policy_documents_size_positive CHECK (file_size_bytes > 0)
);

-- ----------------------------------------------------------------------
-- policy_versions: Each re-extraction or policy update = new version
-- ----------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS policy_versions (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  policy_document_id  uuid NOT NULL REFERENCES policy_documents(id),
  version_number      int NOT NULL,
  is_current          boolean NOT NULL DEFAULT true,
  extraction_job_id   uuid,   -- FK to job_queue.id (populated on job creation)
  extraction_completed_at timestamptz,
  overall_confidence  decimal(4,3),   -- aggregate confidence across all clauses
  clause_count_total  int NOT NULL DEFAULT 0,
  clause_count_auto   int NOT NULL DEFAULT 0,   -- AUTO_ACCEPTED
  clause_count_review int NOT NULL DEFAULT 0,   -- PENDING_REVIEW
  clause_count_unextracted int NOT NULL DEFAULT 0,
  clause_count_absent int NOT NULL DEFAULT 0,
  previous_version_id uuid REFERENCES policy_versions(id),
  created_at          timestamptz NOT NULL DEFAULT now(),

  CONSTRAINT policy_versions_number_positive CHECK (version_number > 0),
  UNIQUE (policy_document_id, version_number)
);
```

```
-- ----------------------------------------------------------------------
-- policy_clauses: One row per extracted clause
-- NOT NULL constraints on citation fields enforce the hallucination guardrail
-- ----------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS policy_clauses (
  id                    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  policy_version_id     uuid NOT NULL REFERENCES policy_versions(id),
  policy_document_id    uuid NOT NULL REFERENCES policy_documents(id),
  clause_type           text NOT NULL,     -- canonical taxonomy value (FAM-01 etc.)
  clause_subtype        text,
  extraction_status     extraction_status NOT NULL,
  confidence_score      decimal(4,3) NOT NULL CHECK (confidence_score BETWEEN 0 AND 1),
  model_a_type          text,
  model_a_confidence    decimal(4,3),
  model_b_type          text,
  model_b_confidence    decimal(4,3),
  model_version_a       text NOT NULL,   -- required: model version logging (S-INV-08)
  model_version_b       text NOT NULL,
  -- HALLUCINATION GUARDRAIL: these three fields are NOT NULL
  supporting_text_span  text NOT NULL,   -- verbatim source text
  source_page           int NOT NULL,
  source_zone           text NOT NULL,
  source_position       jsonb,           -- bounding box if available
  structured_value      jsonb,           -- normalized machine-readable value
  cross_references      uuid[],          -- links to related clause IDs
  unresolved_references jsonb,           -- cross-refs that could not be resolved
  -- Review tracking
  reviewer_id           uuid REFERENCES auth.users(id),
  reviewer_action       review_action,
```

```
  reviewer_notes        text,
  reviewed_at           timestamptz,
  -- Version change tracking (populated for version N+1)
  previous_clause_id    uuid REFERENCES policy_clauses(id),
  version_change_type   version_change_type,
  created_at            timestamptz NOT NULL DEFAULT now(),
  updated_at            timestamptz NOT NULL DEFAULT now(),

  CONSTRAINT clause_citation_complete CHECK (
    extraction_status = 'ABSENT' OR
    (supporting_text_span IS NOT NULL AND length(trim(supporting_text_span)) > 0)
  )
);

-- ----------------------------------------------------------------------
-- clause_review_queue: Human review queue for PENDING_REVIEW clauses
-- ----------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS clause_review_queue (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  clause_id           uuid NOT NULL REFERENCES policy_clauses(id),
  policy_document_id  uuid NOT NULL REFERENCES policy_documents(id),
  priority            text NOT NULL DEFAULT 'STANDARD',   -- STANDARD | HIGH | URGENT
  status              text NOT NULL DEFAULT 'open',        -- open | assigned | resolved
  assigned_to         uuid REFERENCES auth.users(id),
  assigned_at         timestamptz,
  resolved_at         timestamptz,
  resolution_action   review_action,
  sla_deadline        timestamptz,   -- computed from priority at insertion
  created_at          timestamptz NOT NULL DEFAULT now(),

  CONSTRAINT review_queue_unique_open_clause UNIQUE (clause_id)
);

-- ----------------------------------------------------------------------
-- clause_taxonomy_registry: Canonical taxonomy, registered and validated
-- ----------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS clause_taxonomy_registry (
  clause_type         text PRIMARY KEY,
  family_code         text NOT NULL,     -- FAM-01 through FAM-15
  family_name         text NOT NULL,
  description         text NOT NULL,
  downstream_features text[],            -- which feature IDs consume this
  active              boolean NOT NULL DEFAULT true,
  added_at            timestamptz NOT NULL DEFAULT now()
);
```

Migration M-F651-02: Indexes

```
-- ----------------------------------------------------------------------
-- MIGRATION M-F651-02: Indexes for Policy Parsing tables
-- ----------------------------------------------------------------------

-- policy_documents: user lookup and status filtering
CREATE INDEX IF NOT EXISTS idx_policy_docs_user_id
  ON policy_documents(user_id, created_at DESC);

CREATE INDEX IF NOT EXISTS idx_policy_docs_trip_id
  ON policy_documents(trip_id) WHERE trip_id IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_policy_docs_status
  ON policy_documents(status, updated_at DESC);

CREATE INDEX IF NOT EXISTS idx_policy_docs_policy_number
  ON policy_documents(policy_number_encrypted) WHERE policy_number_encrypted IS NOT NULL;

-- policy_versions: current version lookup
CREATE INDEX IF NOT EXISTS idx_policy_versions_doc_id
  ON policy_versions(policy_document_id, version_number DESC);

CREATE INDEX IF NOT EXISTS idx_policy_versions_current
  ON policy_versions(policy_document_id) WHERE is_current = true;
```

```
-- policy_clauses: coverage graph queries (hot path)
CREATE INDEX IF NOT EXISTS idx_clauses_version_type
  ON policy_clauses(policy_version_id, clause_type, extraction_status);

CREATE INDEX IF NOT EXISTS idx_clauses_auto_accepted
  ON policy_clauses(policy_version_id, clause_type) WHERE extraction_status = 'AUTO_ACCEPTED';

CREATE INDEX IF NOT EXISTS idx_clauses_pending_review
  ON policy_clauses(policy_document_id, created_at) WHERE extraction_status = 'PENDING_REVIEW';

-- clause_review_queue: SLA monitoring
CREATE INDEX IF NOT EXISTS idx_review_queue_open
  ON clause_review_queue(priority, sla_deadline) WHERE status = 'open';
```

Migration M-F651-03: RLS Policies

```
-- ---------------------------------------------------------------------
-- MIGRATION M-F651-03: Row-Level Security for Policy Parsing tables
-- All tables follow the same pattern: owner can read/write their data.
-- Support role can see metadata only (not raw document content).
-- No cross-user data exposure at the database layer.
-- ---------------------------------------------------------------------

ALTER TABLE policy_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE policy_versions ENABLE ROW LEVEL SECURITY;
ALTER TABLE policy_clauses ENABLE ROW LEVEL SECURITY;
ALTER TABLE clause_review_queue ENABLE ROW LEVEL SECURITY;

-- policy_documents: user owns their documents
CREATE POLICY "user_owns_documents" ON policy_documents
  FOR ALL TO authenticated
  USING (auth.uid() = user_id);

-- policy_clauses: user can read clauses from their documents
CREATE POLICY "user_reads_own_clauses" ON policy_clauses
  FOR SELECT TO authenticated
  USING (
    policy_document_id IN (
      SELECT id FROM policy_documents WHERE user_id = auth.uid()
    )
  );

-- clause_review_queue: reviewers can see assigned items
CREATE POLICY "reviewer_sees_queue" ON clause_review_queue
  FOR SELECT TO authenticated
  USING (assigned_to = auth.uid() OR status = 'open');

-- Service role bypass for extraction pipeline
-- (extraction runs as service role, not as user)
```

Migration M-F651-04: Event Type Registration

All events emitted by F-6.5.1 must be registered in the event_type_registry per Governance Substrate v1 doctrine:

```
-- ---------------------------------------------------------------------
-- MIGRATION M-F651-04: Register F-6.5.1 event types
-- All event_type values must be registered before emit_event() can use them.
-- ---------------------------------------------------------------------

INSERT INTO event_type_registry (event_type, feature_id, severity, description) VALUES
  ('policy_document_uploaded',    'F-6.5.1', 'info',     'User uploaded a policy document'),
  ('policy_document_rejected',    'F-6.5.1', 'warning',  'Document rejected at intake (type, size,
security)'),
  ('policy_parse_queued',         'F-6.5.1', 'info',     'Extraction job inserted into job_queue'),
  ('policy_parse_started',        'F-6.5.1', 'info',     'Extraction pipeline initiated for
```

```
document'),
  ('policy_parse_stage_complete', 'F-6.5.1', 'info',     'A pipeline stage completed (stage_number,
stage_name)'),
  ('policy_parse_stage_failed',   'F-6.5.1', 'error',    'A pipeline stage failed; extraction may be
degraded'),
  ('policy_parse_complete',       'F-6.5.1', 'info',     'Extraction complete; all clauses
AUTO_ACCEPTED'),
  ('policy_parse_partial',        'F-6.5.1', 'warning',  'Extraction complete; has PENDING_REVIEW
clauses'),
  ('policy_parse_failed',         'F-6.5.1', 'error',    'Extraction failed; document in failed
status'),
  ('clause_review_queued',        'F-6.5.1', 'warning',  'Clause placed in human review queue'),
  ('clause_review_resolved',      'F-6.5.1', 'info',     'Clause review resolved
(accepted/corrected/rejected)'),
  ('clause_review_sla_breach',    'F-6.5.1', 'error',    'Review SLA deadline passed without
resolution'),
  ('policy_version_created',      'F-6.5.1', 'info',     'New policy version extracted (covers
updates/refreshes)'),
  ('policy_coverage_narrowed',    'F-6.5.1', 'warning',  'New version has narrower coverage than
previous version'),
  ('policy_staleness_flagged',    'F-6.5.1', 'warning',  'Platform-maintained guide exceeds refresh
cadence'),
  ('extraction_hallucination_blocked', 'F-6.5.1', 'error', 'Clause blocked from output: missing
source citation'),
  ('model_version_logged',        'F-6.5.1', 'info',     'AI model version recorded for extraction
audit trail')
ON CONFLICT (event_type) DO NOTHING;
```

6. RPC Functions — Canonical API Surface

These are the PostgreSQL RPC functions that define the canonical API surface for F-6.5.1. Application code and the extraction pipeline call these functions — they do not write to tables directly. This ensures all state changes are atomic and logged.

6.1 — initiate_policy_upload()

```
-- Called when user initiates a document upload.
-- Creates the policy_documents row and returns the presigned upload URL.
-- The presigned URL is generated by the application layer, not stored here.

CREATE OR REPLACE FUNCTION initiate_policy_upload(
  p_user_id           uuid,
  p_original_filename text,
  p_file_size_bytes   bigint,
  p_mime_type         text,
  p_storage_key       text,
  p_storage_bucket    text,
  p_trip_id           uuid DEFAULT NULL,
  p_hash_sha256       text DEFAULT NULL
)
RETURNS jsonb LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE
  v_doc_id uuid;
  v_region_check jsonb;
BEGIN
  -- Check mutation guard (Governance Substrate)
  v_region_check := precheck_mutation_guard(
    p_mutation_class := 'evidence_upload'  -- same class as evidence; always allowed
  );

  INSERT INTO policy_documents (
    user_id, original_filename, file_size_bytes, mime_type,
    storage_key, storage_bucket, hash_sha256
  ) VALUES (p_user_id, p_original_filename, p_file_size_bytes, p_mime_type,
    p_storage_key, p_storage_bucket, COALESCE(p_hash_sha256, 'pending')
  ) RETURNING id INTO v_doc_id;

  -- Emit event to ledger (mandatory)
  PERFORM emit_event(
    p_event_type    := 'policy_document_uploaded',
    p_feature_id    := 'F-6.5.1',
    p_scope_type    := 'policy_document',
    p_scope_id      := v_doc_id,
    p_actor_id      := p_user_id,
    p_actor_type    := 'user',
    p_reason_code   := 'user_upload',
    p_resulting_state := jsonb_build_object(
      'status', 'uploaded', 'filename', p_original_filename
    )
  );

  RETURN jsonb_build_object('success', true, 'policy_document_id', v_doc_id);
END; $$;
```

6.2 — submit_clause_for_review() and resolve_clause_review()

```
-- Called by the extraction pipeline when a clause is PENDING_REVIEW.
CREATE OR REPLACE FUNCTION submit_clause_for_review(
  p_clause_id   uuid,
  p_priority    text DEFAULT 'STANDARD'
)
RETURNS jsonb LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE v_doc_id uuid; v_sla timestamptz;
BEGIN
  SELECT policy_document_id INTO v_doc_id FROM policy_clauses WHERE id = p_clause_id;

  v_sla := CASE p_priority
    WHEN 'URGENT'   THEN now() + interval '4 hours'
```

```
    WHEN 'HIGH'     THEN now() + interval '24 hours'
    ELSE                 now() + interval '72 hours'
  END;

  INSERT INTO clause_review_queue (clause_id, policy_document_id, priority, sla_deadline)
  VALUES (p_clause_id, v_doc_id, p_priority, v_sla)
  ON CONFLICT (clause_id) DO NOTHING;

  PERFORM emit_event(
    p_event_type  := 'clause_review_queued',
    p_feature_id  := 'F-6.5.1',
    p_scope_type  := 'policy_clause',
    p_scope_id    := p_clause_id,
    p_actor_type  := 'system',
    p_resulting_state := jsonb_build_object('priority', p_priority, 'sla', v_sla)
  );

  RETURN jsonb_build_object('success', true, 'sla_deadline', v_sla);
END; $$;

-- Called by reviewer to resolve a clause in the review queue.
CREATE OR REPLACE FUNCTION resolve_clause_review(
  p_queue_id      uuid,
  p_reviewer_id   uuid,
  p_action        review_action,
  p_corrected_type text DEFAULT NULL,
  p_notes         text DEFAULT NULL
)
RETURNS jsonb LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE v_clause_id uuid; v_new_status extraction_status;
BEGIN
  SELECT clause_id INTO v_clause_id FROM clause_review_queue WHERE id = p_queue_id;

  v_new_status := CASE p_action WHEN 'REJECTED' THEN 'UNEXTRACTED' ELSE 'AUTO_ACCEPTED' END;

  UPDATE policy_clauses SET
    extraction_status = v_new_status,
    reviewer_id = p_reviewer_id, reviewer_action = p_action,
    reviewer_notes = p_notes, reviewed_at = now(),
    clause_type = COALESCE(p_corrected_type, clause_type)
  WHERE id = v_clause_id;

  UPDATE clause_review_queue SET status = 'resolved', resolved_at = now(),
    resolution_action = p_action WHERE id = p_queue_id;

  PERFORM emit_event(
    p_event_type := 'clause_review_resolved', p_feature_id := 'F-6.5.1',
    p_scope_type := 'policy_clause', p_scope_id := v_clause_id,
    p_actor_id := p_reviewer_id, p_actor_type := 'reviewer',
    p_resulting_state := jsonb_build_object('action', p_action::text, 'new_status',
v_new_status::text)
  );

  RETURN jsonb_build_object('success', true, 'new_status', v_new_status);
END; $$;
```

7. Inputs, Outputs, and Dependencies

7.1 — Inputs

Input	Source	Handling
Policy document (PDF or image)	*User upload or platform-maintained library*	Validated, virus-scanned, hash-verified. Stored in SSE-KMS encrypted object storage per 8.3. Never passed to extraction pipeline as raw file — extracted via storage key by service role only.
Document metadata (filename, claimed type)	*User UI input*	Used to initialize the policy_documents row. Claimed type is a hint only — actual document_type is confirmed by the pipeline during layout analysis.
Trip ID (optional)	*Associated trip context*	Links the extracted policy to a specific trip. If not provided at upload, the user may link it later. Affects Coverage Graph scope.
User ID	*Authenticated session*	Determines ownership, encryption key scope, and row-level security context.
Platform-maintained guide library	*Internal refresh job*	Used for credit card benefit guide extraction. Trigger: quarterly schedule or new version detection.
Amendment / endorsement document	*User upload (separate)*	Detected as amendment type. Pipeline queries for an existing base policy from the same issuer and policy number. If found, amendment is applied. If not found, document is stored and queued for base policy linkage.

7.2 — Outputs

Output	Description & Downstream Consumer
PolicyExtractionResult	The complete extraction output: policy_version_id, extraction status, confidence summary, and the full list of ExtractedClause objects. Consumed by F-6.5.2 (Coverage Graph) to build coverage nodes.
ExtractedClause list (AUTO_ACCEPTED)	The usable, high-confidence clauses. These are the direct inputs to the Coverage Graph node creation process. Each clause is a node candidate with its structured_value, clause_type, and source citation.
Clause review queue items	For every PENDING_REVIEW clause, an entry in clause_review_queue with priority, SLA deadline, and presentation context for the human reviewer.
policy_parse_* ledger events	Immutable event trail for the full extraction lifecycle — queued, started, stage completions, stage failures, complete, partial, failed. Consumed by FOCL Decision Queue for any error-level events.
Policy version diff (for updates)	When a policy is updated, the diff output (EXPANDED/NARROWED/CLARIFIED/RESTRUCTURED changes by clause) is consumed by the notification system and the Coverage Graph update process.
Staleness indicators	For platform-maintained guides that exceed refresh cadence, a staleness flag and date are set on the policy_version. Consumed by coverage display surfaces to show the verification date caveat.

7.3 — Dependencies

Dependency	Type	What F-6.5.1 Needs From It

Section 8.3 — Encryption & Key Management	**Infrastructure REQUIRED**	Object storage SSE-KMS configuration, presigned URL generation, per-user DEK scheme. Without this, document storage cannot meet security requirements.
Section 8.4 — Permissioning & Audit Logs	**Infrastructure REQUIRED**	emit_event() RPC function. precheck_mutation_guard(). event_type_registry (must be seeded with F-6.5.1 event types before pipeline runs). Append-only enforcement.
job_queue table (Migration 00001)	**Schema REQUIRED**	Extraction jobs are inserted into the existing job_queue. The pipeline worker reads from job_queue. No new queue infrastructure required.
event_ledger / event_type_registry (Governance Substrate v1)	**Schema REQUIRED**	All events emitted by F-6.5.1 use emit_event(). M-F651-04 registers the event types. Governance substrate must be applied before F-6.5.1 events can be emitted.
Dual-model AI infrastructure (F-6.5.1 internal)	**Compute REQUIRED**	Two classification models (Model A and Model B) must be deployed and accessible to the extraction pipeline. GPU isolation required per S-INV-08. Models are not third-party APIs — they are platform-deployed to prevent policy data leaving the platform perimeter.
OCR engine (F-6.5.1 internal)	**Compute REQUIRED**	Two OCR engines (primary and secondary for consensus) are required. Standard choices: Tesseract (open source, high control) as primary; cloud OCR API with data processing agreement as secondary. Data must not be retained by the OCR provider.
Pattern library (F-6.5.1 internal)	**Data REQUIRED**	2,400+ clause boundary and citation patterns. This is a maintained data asset. Version-controlled. Must be loaded into the extraction service at startup.
Section 3.4 — Causality Model	**Consuming**	The CCO requires cause category classification. The clause taxonomy intersects with the causality model for Authority-Driven events (FAM-15) and claim routing decisions.
F-6.5.2 — Coverage Graph	**Consuming (downstream)**	Consumes ExtractedClause list to build CoverageNode objects. F-6.5.1 does not depend on F-6.5.2, but F-6.5.2 cannot function without F-6.5.1 output.
F-6.5.5 — Claim Routing & Sequencing	**Consuming (downstream)**	Consumes FAM-12 (filing requirements), FAM-14 (coordination of benefits), and FAM-11 (general exclusions) to determine routing sequence.
Section 7.3 — Screen Surface Registry	**UX (consuming)**	Upload flow screen, parsing progress screen, review queue screen. These surfaces must exist to support the user-facing upload and status tracking flows.
Section 9.2 — Confidence Labels & Uncertainty Language	**UX (consuming)**	Confidence score display to users. The language rules for communicating extraction confidence levels (HIGH / PENDING / UNEXTRACTED) are defined in 9.2, not here.

8. Failure Modes & Safe Defaults

8.1 — Pipeline Failure Modes

Failure Condition	What the System Does	What the User Sees	Downstream Effect
OCR confidence below minimum on all pages	Set status = needs_review. Preserve raw text. Emit policy_parse_partial. Route all clauses to PENDING_REVIEW with HIGH priority.	*"A few sections of your document are being manually verified. We'll let you know when they're ready."*	Coverage Graph excludes this document until review complete. No guidance served from unverified data.
Dual models disagree on majority of clauses	Set status = needs_review. Route disagreed clauses to PENDING_REVIEW. Accept agreed clauses normally.	*Same as above. Agreed clauses are available; disputed clauses show as PENDING.*	Partial coverage graph possible from agreed clauses. Disputed categories shown as PENDING in UI.
Document structure unrecognized (rare layout)	Emit policy_parse_failed with failure_code = structure_unrecognized. Retain the raw document. Flag for manual extraction.	*"We weren't able to automatically read this document. Your document is safely stored. A team member will review it."*	No coverage data until manual extraction complete. Trip risk score shows gap indicator for this document.
Hallucination guardrail triggered (clause has no citation)	Block clause from output. Emit extraction_hallucination_blocked. Mark clause as UNEXTRACTED.	*No user-visible change. The clause simply does not appear. Coverage category shows UNEXTRACTED indicator.*	Downstream features treat the category as UNEXTRACTED. Never served as coverage data.
Job queue worker unavailable	Job remains in pending state. Retry per job_queue.max_retries (default: 3). Document shows status = queued.	*"Your document is being processed. This may take a few minutes."*	No downstream effect until job runs. Upload confirmation is already given.
Model unavailable (AI service down)	Stage 5 fails. Emit policy_parse_stage_failed. Document status = needs_review. Queue for retry when model recovers.	*Same as processing message. Status does not show error to user unless extended delay (>30 min) triggers notification.*	Extraction paused. No partial output served. Consistency preserved.
Malicious document detected (virus scan)	Reject immediately. Do not create policy_documents row. Emit policy_document_rejected with failure_code = security_scan_failed.	*"We couldn't process this file. Please try uploading a different version, or contact support."*	No data created. No downstream effect. Audit log preserved.

8.2 — The Safe Default Rule

When in doubt about a clause, the platform defaults to UNEXTRACTED — not to assuming coverage.

- An UNEXTRACTED clause means the platform cannot confirm presence or absence of that coverage type. The user is told this clearly.
- An ABSENT clause means extraction confirmed the coverage type is not present in the document.
- Neither UNEXTRACTED nor ABSENT tells the user "you are not covered." They tell the user "we cannot confirm from this document."
- The distinction between ABSENT and UNEXTRACTED is preserved in the Coverage Graph and shown to the user differently.
- This applies even under surge conditions. Extraction accuracy does not degrade gracefully toward false positives — it fails toward UNEXTRACTED or PENDING_REVIEW.

9. Security Requirements (Section 8.6.1 Compliance)

This section declares F-6.5.1's compliance with each of the 14 security requirements from 8.6.1. All 14 must be declared — silence is not compliance.

Req	Requirement	F-6.5.1 Declaration
S-01	**Data classification**	T2 Sensitive (policy documents, policy numbers). Policy_number field encrypted at field level per 8.3.4. storage_key (pointer to object storage) is metadata — does not contain sensitive data.
S-02	**Encryption at rest**	All policy_documents rows are standard encrypted-at-rest (database-level AES-256). Policy document files are stored in SSE-KMS encrypted object storage with per-user keys per 8.3.3. policy_number_encrypted contains ciphertext only.
S-03	**Encryption in transit**	All upload and download traffic uses TLS 1.3. Presigned URLs are delivered over HTTPS. Service-to-service calls from extraction pipeline to object storage use HTTPS with service role credentials.
S-04	**Authentication & authorization**	Upload: requires authenticated user session. Extraction pipeline: runs as service role (not as user). Clause review: requires authenticated reviewer session. Step-up verification required for bulk document deletion or bulk status changes by support role.
S-05	**PII exposure**	policy_number is PII. It is stored in policy_number_encrypted (ciphertext). Plaintext policy_number is displayed only to the account owner in authenticated session, never to support role, never in logs. storage_key is never logged or displayed.
S-06	**Object storage access**	Extraction pipeline accesses object storage via service role only. No long-lived credentials. Presigned download URLs for user access: TTL 15 minutes. No presigned URLs with longer TTL for T2 documents.
S-07	**Audit trail coverage**	All state changes in policy_documents (status transitions), clause_review_queue resolutions, and extraction job outcomes emit events to event_ledger via emit_event(). If emit_event() fails, the triggering action is rolled back (ledger-first enforcement per 8.4).
S-08	**AI/LLM boundary**	Both classification models run in isolated compute context. Context is scoped to the current extraction job only. model_version_a and model_version_b are stored on every policy_clause row per S-INV-08. No model has write access to any database table.
S-09	**Failure mode**	See Section 8. Failures route to PENDING_REVIEW or UNEXTRACTED — never to false positive coverage assertions. User messaging per 7.2 Calm Mode Language System. No silent failures.
S-10	**Surge behavior**	Under ELEVATED mode: extraction continues normally. Under PROTECTIVE mode: new document uploads accepted (evidence capture always available per 8.4 guarantee), but extraction jobs are queued and do not execute until mode returns to NORMAL or ELEVATED. Status shown to user as "Processing will resume shortly."
S-11	**Data retention**	policy_documents: T2 retention schedule (8.2.4). Active subscriber: 3-5 years. Post-cancellation: 12-24 months. policy_clauses: same retention as the parent document. clause_review_queue: retained as audit record for 5 years after resolution.
S-12	**Third-party exposure**	OCR secondary engine: data transmitted only with a data processing agreement prohibiting retention. No other third-party services receive policy document content. The two classification models are platform-deployed.
S-13	**HIPAA applicability**	Policy documents do not inherently contain PHI. If a user uploads a document that contains health information (e.g., medical evacuation policy), the document is stored in T2 tier. FAM-05 and FAM-06 clauses are flagged as health-adjacent. HIPAA-aligned access controls apply.
S-14	**Export profile binding**	Policy document downloads are routed through the Centralized Export Service (8.7). Export profile: TRAVELER_PERSONAL. policy_number is included in redacted form (last 4 digits only) in any export not explicitly authorized by the user.

10. Telemetry, QA Scenarios, and Stress Tests

10.1 — What We Measure

Metric	Target	Why It Matters
Clause-level extraction accuracy (human-validated sample)	**≥ 99.0%**	The core product promise. Measured monthly on a stratified sample of 500 clauses drawn from AUTO_ACCEPTED output, reviewed by a human annotator blind to model output.
Document-level completion rate	**≥ 95%**	Percentage of submitted documents that reach either "complete" or "partial" status (not "failed" or "needs_review" without resolution). Failures require investigation.
AUTO_ACCEPTED rate (all documents)	**≥ 75%**	What percentage of extracted clauses are high-confidence without human review. Below 75% indicates model drift or a new document type the models haven't seen.
Human review queue SLA adherence	**≥ 99%**	HIGH priority items reviewed within 24 hours. STANDARD within 72 hours. SLA breach events trigger FOCL Decision Queue item.
Median extraction time (queued → complete)	**< 3 minutes**	User-facing expectation. Measured from job_queue insertion to policy_document status = complete.
Hallucination guardrail triggers per 1,000 clauses	**< 1**	Every trigger is a clause that would have been served without a source citation. Triggers are investigated individually.
Model version drift (days since last model update)	**< 90 days**	Models are retrained on new labeled data quarterly. Stale models lose accuracy on new policy structures.
Cross-reference resolution rate	**≥ 80%**	Percentage of clause cross-references successfully resolved. Low rates indicate structural changes in common policy formats.

10.2 — QA Scenarios (Pressure Tests)

QA-01: Scanned PDF, low resolution	Upload a 100 DPI scan of a 40-page travel policy. Expected: pipeline detects scan quality issue, applies secondary OCR, produces extraction with elevated PENDING_REVIEW count. No false AUTO_ACCEPTED clauses with citation from garbled OCR output.
QA-02: Policy with amendment sheet	Upload a base policy + a separate endorsement that narrows baggage coverage. Expected: endorsement detected, change_type = NARROWED on baggage clause, user notification triggered, previous version preserved.
QA-03: Multi-column layout	Upload a policy with two-column layout and a sidebar. Expected: columns are extracted in correct reading order (left column top-to-bottom, then right column), not left-right line by line. No clause boundary errors at column transitions.
QA-04: Dual-model disagreement on exclusion	Submit a clause where Model A classifies as FAM-11 (General Exclusion) and Model B classifies as FAM-02 (Travel Delay). Expected: confidence penalty applied, clause routed to PENDING_REVIEW with both classifications shown to reviewer. Not served to Coverage Graph until resolved.
QA-05: Policy number in plaintext logs	During extraction, trigger an error that produces a full traceback. Expected: policy_number does not appear in any log line, error message, or audit event payload. Only policy_number_encrypted (ciphertext) is present in stored records.
QA-06: Hallucination guardrail enforcement	Inject a clause into the extraction pipeline with a non-null clause_type but a null supporting_text_span. Expected: database constraint violation is raised, clause is not inserted, hallucination_blocked event is emitted.
QA-07: PROTECTIVE	Put region in PROTECTIVE mode. Upload a policy document. Expected: upload is

mode surge	accepted (evidence capture never blocked). Extraction job is inserted into job_queue but not executed until mode changes. Status shows "Processing will resume shortly."
QA-08: Credit card guide version update	Platform-maintained guide is updated by the issuer. New version is fetched. Expected: new version is extracted, diff is computed, if any benefit is NARROWED, the change is logged and FOCL is notified. Old version is preserved with its extraction.
QA-09: Cross-reference chain resolution	Policy has: benefit clause in FAM-02 that defines "covered event" by reference to FAM-13 (definitions), which in turn references a government advisory requirement. Expected: full chain is resolved and linked in cross_references. Unresolved chains are logged in unresolved_references.
QA-10: 99% accuracy validation run	Human annotator reviews 500 randomly sampled AUTO_ACCEPTED clauses from production (blind to model output). Expected: ≥ 495 of 500 (99.0%) are correctly classified. Any failure triggers immediate model investigation before next extraction cycle.

10.3 — Section 15 Stress Test Integration

F-6.5.1 must be included in the Section 15 Stress Testing Framework. The following scenarios are added to the platform-wide stress test battery:

Surge: 10,000 simultaneous uploads	All upload flow paths, object storage write pressure, job_queue insertion volume, presigned URL generation rate. Expected: all uploads accepted, no data loss, extraction jobs queued in order, no presigned URL TTL violations.
Worker exhaustion	All job_queue workers are unavailable for 2 hours. Expected: jobs accumulate in pending state, user sees "processing" status, no jobs lost, no retry exhaustion for reasonable accumulation window. Workers resume and drain queue without errors.
Model service unavailable	AI classification service returns 503 for 30 minutes. Expected: pipeline stage 5 fails cleanly per failure mode 8.1, documents enter needs_review status, FOCL Decision Queue receives error events, no partial or incorrect extractions are served.
Corrupt document surge	100 documents with SHA-256 mismatch (corruption detected post-upload). Expected: all 100 trigger policy_parse_stage_failed, documents enter failed status, no extraction attempted, audit events emitted, no data served.

11. Feature Invariant Register (F-6.5.1)

These invariants are binding. Any system state in which one of these is violated is a defect to be corrected immediately, regardless of the cause.

Invariant	Rule
F651-I-01	Every ExtractedClause with status AUTO_ACCEPTED or PENDING_REVIEW has a non-null, non-empty supporting_text_span referencing its source document. No exceptions. Enforced by CHECK constraint.
F651-I-02	No clause with extraction_status = UNEXTRACTED or PENDING_REVIEW is ever served to the Coverage Graph, Claim Routing engine, or any user-facing guidance surface. ABSENT clauses are served as absence indicators — not as coverage.
F651-I-03	model_version_a and model_version_b are never null on any policy_clause row. Both model versions are recorded at extraction time, even if the models agree.
F651-I-04	Every state transition in policy_documents (status changes) and every clause_review_queue resolution emits an event to the event_ledger via emit_event(). If the emit fails, the state transition is rolled back.
F651-I-05	policy_number is never stored in plaintext in the database. Only policy_number_encrypted (AES-256 envelope-encrypted ciphertext) is persisted.
F651-I-06	Policy document files are accessible only via service-role presigned URLs with TTL ≤ 15 minutes. No permanent storage URLs are generated or exposed.
F651-I-07	A clause type is only assigned from the clause_taxonomy_registry. Ad-hoc or free-text clause types are rejected at the application layer.
F651-I-08	When a new policy version is extracted, the previous version's policy_clauses are preserved (immutable, not updated). Version history is append-only.
F651-I-09	Under PROTECTIVE mode, new extraction jobs are queued but not executed. Documents already in processing are completed. No extraction job is silently abandoned.
F651-I-10	The dual-model validation is not optional. Single-model extraction is not permitted in production, even for document types where one model is known to perform better.
F651-I-11	Clause review SLA breaches emit a clause_review_sla_breach event to the event_ledger. This event must appear in the FOCL Decision Queue. SLA breaches are not silent.
F651-I-12	The platform-maintained credit card benefit guide library is refreshed no less than quarterly per guide. Guides exceeding 120 days without refresh display a staleness indicator on all surfaces that show their data.
F651-I-13	No policy document content (beyond metadata) is transmitted to any third-party service without a signed data processing agreement that prohibits retention of that content.
F651-I-14	ABSENT and UNEXTRACTED are distinct states with distinct meanings and distinct user-facing presentation. They must never be conflated in the database, the Coverage Graph, or the UI.